Prompt: Create a C++ program that reads a text file line by line executes the commands on each line to prioritize emails. The emails need to be stored in a list-based Max heap. The emails are prioritized from most important sender to least important sender: Boss, Subordinate, Peer, ImportantPerson, OtherPerson. The user will input a text file with each line having a list of either four commands: EMAIL (which is followed by the sender category, subject line, and date, each separated only by commas except the EMAIL command which is separated by a space)(The EMAIL command adds the email and the extra information into the Max Heap), NEXT (which prints the newest most important email's information), READ (which removes the newest most important email from the Max Heap, adjusts the Max Heap accordingly, and prints a message saying the CEO read it), and COUNT (Which prints how many emails are in the Max Heap to still be read). The text file's commands are separated by new lines. The EMAIL command has the sender category, subject line, and date on the same line as the EMAIL text command. Read and perform the commands in the text file line-by-line.

Gemini Code (prompted online on personal computer (gemini pro)):

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

// Enumeration for sender priorities (Higher value = Higher priority)
enum SenderPriority {
    OTHER_PERSON = 1,
    IMPORTANT_PERSON = 2,
    PEER = 3,
    SUBORDINATE = 4,
    BOSS = 5
};

// Map string category to enum priority
SenderPriority getPriority(const std::string& category) {
    if (category == "Boss") return BOSS;
    if (category == "Subordinate") return SUBORDINATE;
    if (category == "Peer") return PEER;
    if (category == "ImportantPerson") return IMPORTANT_PERSON;
    return OTHER_PERSON;
}

// Structure to represent an Email
struct Email {
    std::string category;
    std::string subject;
```

```
    std::string date;
    SenderPriority priority;
    long long sequenceId; // Ensures "newest" breaks ties for the same sender type

    // Overloaded operators for heap comparisons
    bool operator<(const Email& other) const {
        if (this->priority != other.priority) {
            return this->priority < other.priority;
        }
        // If priorities are equal, the newer email (higher sequence ID) takes precedence
        return this->sequenceId < other.sequenceId;
    }

    bool operator>(const Email& other) const {
        if (this->priority != other.priority) {
            return this->priority > other.priority;
        }
        return this->sequenceId > other.sequenceId;
    }
};

// Max Heap implementation using a vector/list
class EmailMaxHeap {
private:
    std::vector<Email> heap;

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[index] > heap[parent]) {
                std::swap(heap[index], heap[parent]);
                index = parent;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = heap.size();
        while (2 * index + 1 < size) {
            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;
            int largest = index;
```

```
            if (heap[leftChild] > heap[largest]) {
                largest = leftChild;
            }
            if (rightChild < size && heap[rightChild] > heap[largest]) {
                largest = rightChild;
            }

            if (largest != index) {
                std::swap(heap[index], heap[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }

public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    bool isEmpty() const {
        return heap.empty();
    }

    Email getMax() const {
        if (!heap.empty()) {
            return heap[0];
        }
        throw std::runtime_error("Heap is empty");
    }

    void extractMax() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    size_t size() const {
```

```
        return heap.size();
    }
};

// Helper function to trim whitespaces from parsed CSV strings
std::string trim(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == std::string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

int main() {
    std::string filename;
    std::cout << "Enter the command text file name: ";
    std::cin >> filename;

    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error: Could not open file " << filename << std::endl;
        return 1;
    }

    EmailMaxHeap emailHeap;
    long long globalSequenceCounter = 0;
    std::string line;

    while (std::getline(file, line)) {
        if (line.empty()) continue;

        std::stringstream ss(line);
        std::string command;
        ss >> command;

        if (command == "EMAIL") {
            std::string remaining, category, subject, date;
            std::getline(ss, remaining); // Get the rest of the line after "EMAIL "

            std::stringstream csvStream(remaining);
            std::getline(csvStream, category, ',');
            std::getline(csvStream, subject, ',');
            std::getline(csvStream, date, ',');

            category = trim(category);
```

```
            subject = trim(subject);
            date = trim(date);

            Email newEmail = {
                category,
                subject,
                date,
                getPriority(category),
                ++globalSequenceCounter
            };
            emailHeap.insert(newEmail);

        } else if (command == "NEXT") {
            if (emailHeap.isEmpty()) {
                std::cout << "No emails in the queue.\n";
            } else {
                Email top = emailHeap.getMax();
                std::cout << "Next Priority Email: [" << top.category << "] "
                          << top.subject << " (Received: " << top.date << ")\n";
            }

        } else if (command == "READ") {
            if (emailHeap.isEmpty()) {
                std::cout << "No emails to read.\n";
            } else {
                Email top = emailHeap.getMax();
                emailHeap.extractMax();
                std::cout << "The CEO read the email from " << top.category
                          << " regarding \"" << top.subject << "\".\n";
            }

        } else if (command == "COUNT") {
            std::cout << "Emails remaining in Max Heap: " << emailHeap.size() << "\n";
        }
    }

    file.close();
    return 0;
}
```

Chatgpt's code (also prompted on their website, on personal computer):

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

// ---------------------------------------------------------
// Email structure
// ---------------------------------------------------------
struct Email
{
    string senderCategory;
    string subject;
    string date;
};

// ---------------------------------------------------------
// Node for the linked list
// ---------------------------------------------------------
struct Node
{
    Email email;
    Node* next;

    Node(const Email& e)
    {
        email = e;
        next = nullptr;
    }
};

// ---------------------------------------------------------
// List-based Max Heap
// ---------------------------------------------------------
class MaxHeap
{
private:
    Node* head;
    int heapSize;

    // Returns the node at a particular heap index.
    // The heap is represented in level-order in the linked list.
```

```
Node* getNode(int index)
{
    Node* current = head;

    for (int i = 0; i < index && current != nullptr; i++)
    {
        current = current->next;
    }

    return current;
}

// Returns the priority associated with a sender category.
int getPriority(const string& category)
{
    if (category == "Boss")
        return 5;
    else if (category == "Subordinate")
        return 4;
    else if (category == "Peer")
        return 3;
    else if (category == "ImportantPerson")
        return 2;
    else if (category == "OtherPerson")
        return 1;

    return 0;
}

// Determines whether email1 has higher priority than email2.
bool higherPriority(const Email& email1, const Email& email2)
{
    int priority1 = getPriority(email1.senderCategory);
    int priority2 = getPriority(email2.senderCategory);

    // Sender category has the highest priority.
    if (priority1 != priority2)
    {
        return priority1 > priority2;
    }

    // If sender categories are equal, newer date wins.
    // This assumes dates are entered in a sortable format
    // such as YYYY-MM-DD.
```

```
    return email1.date > email2.date;
}

// Swaps the email data stored in two nodes.
void swapEmails(Node* a, Node* b)
{
    Email temp = a->email;
    a->email = b->email;
    b->email = temp;
}

// Restores the Max Heap property going downward.
void heapifyDown(int index)
{
    while (true)
    {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
        int largest = index;

        Node* current = getNode(index);
        Node* left = getNode(leftChild);
        Node* right = getNode(rightChild);

        if (current == nullptr)
            return;

        if (left != nullptr &&
            higherPriority(left->email, current->email))
        {
            largest = leftChild;
        }

        Node* largestNode = getNode(largest);

        if (right != nullptr &&
            higherPriority(right->email, largestNode->email))
        {
            largest = rightChild;
        }

        if (largest == index)
            break;
```

```
            swapEmails(current, getNode(largest));

            index = largest;
        }
    }

    // Restores the Max Heap property going upward.
    void heapifyUp(int index)
    {
        while (index > 0)
        {
            int parentIndex = (index - 1) / 2;

            Node* current = getNode(index);
            Node* parent = getNode(parentIndex);

            if (current == nullptr || parent == nullptr)
                return;

            if (higherPriority(current->email, parent->email))
            {
                swapEmails(current, parent);
                index = parentIndex;
            }
            else
            {
                break;
            }
        }
    }

public:

    MaxHeap()
    {
        head = nullptr;
        heapSize = 0;
    }

    ~MaxHeap()
    {
        clear();
    }
```

```
// ----------------------------------------------------
// Add an email to the heap
// ----------------------------------------------------
void insert(const Email& email)
{
   Node* newNode = new Node(email);

   if (head == nullptr)
   {
      head = newNode;
   }
   else
   {
      Node* last = getNode(heapSize - 1);
      last->next = newNode;
   }

   heapSize++;

   // Restore Max Heap property.
   heapifyUp(heapSize - 1);
}

// ----------------------------------------------------
// Return the highest-priority email without removing it.
// ----------------------------------------------------
Email* peek()
{
   if (head == nullptr)
      return nullptr;

   return &head->email;
}

// ----------------------------------------------------
// Remove and return the highest-priority email.
// ----------------------------------------------------
Email* removeMax()
{
   if (head == nullptr)
      return nullptr;

   // Save the root email.
   Email* result = new Email(head->email);
```

```
    // If there is only one email.
    if (heapSize == 1)
    {
        delete head;
        head = nullptr;
        heapSize = 0;

        return result;
    }

    // Find the last node.
    Node* last = getNode(heapSize - 1);

    // Move the last email to the root.
    head->email = last->email;

    // Find the node before the last node.
    Node* previous = getNode(heapSize - 2);

    previous->next = nullptr;
    delete last;

    heapSize--;

    // Restore Max Heap property.
    heapifyDown(0);

    return result;
}

// ------------------------------------------------
// Return number of unread emails.
// ------------------------------------------------
int count()
{
    return heapSize;
}

// ------------------------------------------------
// Delete all nodes.
// ------------------------------------------------
void clear()
{
```

```
        Node* current = head;

        while (current != nullptr)
        {
            Node* next = current->next;
            delete current;
            current = next;
        }

        head = nullptr;
        heapSize = 0;
    }
};

// ------------------------------------------------------
// Remove leading/trailing whitespace from a string.
// ------------------------------------------------------
string trim(const string& str)
{
    size_t first = str.find_first_not_of(" \t\r\n");
    size_t last = str.find_last_not_of(" \t\r\n");

    if (first == string::npos)
        return "";

    return str.substr(first, last - first + 1);
}

// ------------------------------------------------------
// Process one command from the input file.
// ------------------------------------------------------
void processCommand(const string& line, MaxHeap& emailHeap)
{
    if (line.empty())
        return;

    // EMAIL command has a space after EMAIL.
    if (line.rfind("EMAIL ", 0) == 0)
    {
        string emailData = line.substr(6);

        stringstream ss(emailData);

        string senderCategory;
```

```
    string subject;
    string date;

    // EMAIL sender,subject,date
    getline(ss, senderCategory, ',');
    getline(ss, subject, ',');
    getline(ss, date);

    senderCategory = trim(senderCategory);
    subject = trim(subject);
    date = trim(date);

    Email email;
    email.senderCategory = senderCategory;
    email.subject = subject;
    email.date = date;

    emailHeap.insert(email);

    cout << "Email added: "
         << senderCategory << ", "
         << subject << ", "
         << date << endl;
}

else if (line == "NEXT")
{
    Email* email = emailHeap.peek();

    if (email == nullptr)
    {
        cout << "No emails to read." << endl;
    }
    else
    {
        cout << "Next email:" << endl;
        cout << "Sender: " << email->senderCategory << endl;
        cout << "Subject: " << email->subject << endl;
        cout << "Date: " << email->date << endl;
    }
}

else if (line == "READ")
{
```

```
        Email* email = emailHeap.removeMax();

        if (email == nullptr)
        {
            cout << "No emails to read." << endl;
        }
        else
        {
            cout << "CEO read the email: "
                 << email->subject << endl;

            delete email;
        }
    }

    else if (line == "COUNT")
    {
        cout << "Unread emails: "
             << emailHeap.count() << endl;
    }

    else
    {
        cout << "Unknown command: " << line << endl;
    }
}

// ----------------------------------------------------------
// Main
// ----------------------------------------------------------
int main()
{
    string fileName;

    cout << "Enter the name of the input file: ";
    getline(cin, fileName);

    ifstream inputFile(fileName);

    if (!inputFile.is_open())
    {
        cout << "Error: Could not open file." << endl;
        return 1;
    }
```

```
    MaxHeap emailHeap;

    string line;

    // Read and execute commands one line at a time.
    while (getline(inputFile, line))
    {
        line = trim(line);

        if (!line.empty())
        {
            processCommand(line, emailHeap);
        }
    }

    inputFile.close();

    return 0;
}
```

COMPARISON:

Notable comparisons (code structure)

	Gemini	Chatgpt
dates	gemini's code does not have specific date requirements, most likely because it does not use the date for priority like chatgpt does.	chatgpt exclusively uses year-month-day systems. its not particularly bad but i dislike it a lot.
priority	Gemini creates an enum group, defining each category as a set number. Has a separate function that reads the string in the email to assign one of those priority categories to it. Gemini creates a global counter which tracks priority in case two emails are the same.	chatgpt uses a function with multiple if-statements to return a set number if one of the categories is used. if two emails have the same priority, chatgpt's code uses the date sent to make one have higher priority. this is ineffective if two emails of equal priority were sent on the same day

Heapify up/down	Gemini's prompt no longer uses recursion to heapify down. In the previous assignment, the recursive statement increased space complexity, but in this result it is now a while loop which decreases **space complexit**y.	chatgpt uses a while loop to heapify down, consistently changing a variable that instructs where to use the code until the break line is used. this is identical to gemini's.
main	Gemini's main is longer. I believe some of what chatgpt does in functions, gemini gets done in the main, like taking in the command and calling functions off of that. moving this into its own function is a possibility but might mess with specific variables that are defined in main (because object oriented programming).	chatgpts main is smaller and seems to focus on calling whatever functions are necessary for each line. having a smaller main that exclusively calls functions may help maintainability.
comments	Gemini gives fairly helpful comments. Its not a lot, but makes the code easier to follow.	gemini does slightly better than chatgpt on comments, more notability large —- brackets to categorize, but still does not do very well in the comment depatment. i do like how they started to comment more if statements and smaller blocks of code, but its still not that much
SPACE	Gemini's space complexity depends on the amount of emails input, so itll be O(n). minor notice that having a string for the category of each can increase the space and time complexity slightly. this can be changed to reduce this.	chatgpts space complexity is just as gemini's so itll be O(n).
TIME	depends mainly on heapify functions which run on Olog(n) minor notice that having a string for the category of each can increase the space and time complexity slightly. this can be changed to reduce this.	chatgpts time complexity is near identical to Gemini's.

CHOICE: Gemini prompt

reasoning: better blocks of code and more object sections for formatting. specific date structure unneeded, and global counter for equal priority is more correct. chatgpt's code does handle commands in a separate function, which i may try to incorporate into gemini's code later.

- **Modifying code (what do i plan on changing?)**
    - having the boss read a file tells the user what file was read. not supposed to be done as per the the requirements. remove
    - move command list into separate function for maintsinability. ✅
    -
    - change category string to enum to reduce memory allocation and thus space and time complexity.
    -
    - Comments - a better idea of what each function and block of code does is essential in keeping maintainability. Gemini's prompt gives a good start, but could always be more descriptive. ✅
    -
    - gemini throws a runtime error if you try to get the root if there is nothing in the list. i do not want the code to end at this moment, so potentially adding try/except will be beneficial *(added note: after rereading the code, there are if statements around whatever is used that calls this function to check and do different code if the heap is empty, so this should not be an issue)* 🟡
    -
    - making sure the identified potential issues (like using two reads in a row or next and read when there are no more emails in the code) can be handled (read above note) ✅

CHANGED CODE:

```
//
//Name of program: 3228668_Warren_assignment3.c
//Discription: A c++ program that takes in a list of commands holding
emails,
//adds them to a max heap by priority, and completes each list of
commands.
//commands able to be used:
//EMAIL (category, subject, date)
//READ
//COUNT
//NEXT
//
//Inputs: .txt file where each line contains only one of any of the above
mentioned commands.
```

```
// Output: all commands completed (printed statements that commands were
done)
// collaborators: none
//other sources: Google Gemini's code, chatgpt's code for pros and cons
//Author: Christopher Warren
//Creation date: 10/1/2026
// Revision date: 10/1/2026
// Revisions:1. added comments, 2: moved command if statements to functon
runcommands(), 3: email category removed, only uses priority enum, 4:
fixed print statements


#include <iostream> //input/output
#include <fstream> //reading/writing files
#include <sstream> //insertion/extraction
#include <string> //strings
#include <vector> //vectors


// Enumeration for sender priorities
// (Higher value = Higher priority)
// Boss is highest, Other Person is lowest
enum SenderPriority {
    OTHER_PERSON = 1,
    IMPORTANT_PERSON = 2,
    PEER = 3,
    SUBORDINATE = 4,
    BOSS = 5
};

// Map string category to enum priority
SenderPriority getPriority(const std::string& category) {
    if (category == "Boss") return BOSS;
    if (category == "Subordinate") return SUBORDINATE;
    if (category == "Peer") return PEER;
    if (category == "ImportantPerson") return IMPORTANT_PERSON;
    return OTHER_PERSON;
}

//Email object Structure to represent an Email
```

```cpp
// Contains category, subject, and date inclueded in commands
// adds global counter value for extra priority

struct Email {
    //std::string category;
    std::string subject;
    std::string date;
    SenderPriority priority;
    long long sequenceId; // Ensures "newest" breaks ties for the same
sender type

    // Overloaded operators for heap comparisons
    bool operator<(const Email& other) const {
        //Returns if one has higher priority than the other
        if (this->priority != other.priority) {
            return this->priority < other.priority;
        }
        // If priorities are equal, the newer email (higher sequence ID)
takes precedence
        return this->sequenceId < other.sequenceId;
    }

    bool operator>(const Email& other) const {
        if (this->priority != other.priority) {
            return this->priority > other.priority;
        }
        return this->sequenceId > other.sequenceId;
    }
};

std::string prioritytostr(SenderPriority sp){
    if (sp == BOSS) return "Boss";
    if (sp == SUBORDINATE) return "Subordinate";
    if (sp == PEER) return "Peer";
    if (sp == IMPORTANT_PERSON) return "ImportantPerson";
    return "OtherPerson";
}

// Max Heap implementation using a vector/list
class EmailMaxHeap {
```

```
private:
    std::vector<Email> heap; // heap is kept as a vector

    //heapify up to correct heap structure
    //uses for while loop instead of recursion like last time
    //inputs: index of email
    //void - no output
    void heapifyUp(int index) {
        while (index > 0) { // while not at root
            int parent = (index - 1) / 2; //get parent
            if (heap[index] > heap[parent]) { //if the child is higher
priority than parent
                std::swap(heap[index], heap[parent]); //swap
                index = parent; //move index to where the child went
            } else {
                break; //end loop
            }
        }
    }
    //heapify down (while loop)
    //input: index of email
    //no output
    void heapifyDown(int index) {
        int size = heap.size(); //get size of heap
        while (2 * index + 1 < size) { // while there is at least once
child
            int leftChild = 2 * index + 1; //left child
            int rightChild = 2 * index + 2; // right child
            int largest = index; // parent starts as largest

            if (heap[leftChild] > heap[largest]) {
                largest = leftChild; //we start with left child to check
if laargest.
                //if it is larger than parent, we mark it the largest
            }
            if (rightChild < size && heap[rightChild] > heap[largest]) {
                largest = rightChild; //now the right child is checked if
largest
                // if it is larger than the oarent, we mark it as the
largest
```

```
            }

            if (largest != index) { // if either of the children were made
to be larger
                std::swap(heap[index], heap[largest]); //swap
                index = largest; //move the index to the smaller value
that was swapped and repeat
            } else {
                break; //list is sorted
            }
        }
    }

public:
    // insert function
    //inserts an email object to heap and calls heapify up to correct list
    // input: email object
    // no output, adds and fixes heap with new email
    void insert(const Email& email) {
        heap.push_back(email); //appends email to heap at the end
        heapifyUp(heap.size() - 1); //heapify up at the last index (where
new email is)
    }

    bool isEmpty() const { // checks if heap is empty (bool)
        return heap.empty(); // if empty, true
    }

    Email getMax() const { //returns root email object (highest priority),
no input
        if (!heap.empty()) {
            return heap[0];
        }
        throw std::runtime_error("Heap is empty"); // if heap is empty,
throw error (BAD)
    }

    void extractMax() { // returns root email object and removes it
        if (heap.empty()) return;
```

```
        heap[0] = heap.back(); // takes the last email in list amd makes
it to first
        heap.pop_back(); // removes duplicate last obj at the end
        if (!heap.empty()) {
            heapifyDown(0); // heapify down  once the last object was
added to the first
        }
    }

    // returns heap size
    size_t size() const {
        return heap.size();
    }
};

// Helper function to trim whitespaces from parsed CSV strings
std::string trim(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == std::string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1)); //output is string
without whitespaces
}

//process command handles if statements for the commands in the user file
//inputs: command pointer
// ss pointer (separate parts of line)
// email heap pointer
//global counter pointer
//no outputs but processes commands EMAIL, NEXT, READ, and COUNT
void processCommand(const std::string& command,
                    std::stringstream& ss,
                    EmailMaxHeap& emailHeap,
                    long long& globalSequenceCounter){
        if (command == "EMAIL") { // email actions
            std::string remaining, category, subject, date; // declaring
everything included in email line
            std::getline(ss, remaining); // Get the rest of the line after
"EMAIL "
```

```
            std::stringstream csvStream(remaining); // we separate
everything after email to categorize
            std::getline(csvStream, category, ','); // category here, will
become priority later
            std::getline(csvStream, subject, ','); // subject header here
            std::getline(csvStream, date, ','); // date here

            category = trim(category); //get priority for this email
            subject = trim(subject);
            date = trim(date);

            Email newEmail = { //we make a new email object for this
information
                //category,
                subject,
                date,
                getPriority(category),   //get priority for this email
                ++globalSequenceCounter  //global counter value for extra
priority
            };
            emailHeap.insert(newEmail); //once complete, put email in heap

        } else if (command == "NEXT") { //next command displays what is in
the root of the maxheap
            if (emailHeap.isEmpty()) { //does not show root if there is
nothing in it
                std::cout << "No emails in the queue.\n\n";
            } else {
                Email top = emailHeap.getMax();
                std::cout << "Next Priority Email: \n\tSender: " <<
prioritytostr(top.priority) << "\n\tSubject: "
                          << top.subject << "\n\tDate: " << top.date <<
"\n\n"; // displays info
            }

        } else if (command == "READ") { //the boss will read the next
email
            if (emailHeap.isEmpty()) { // if no emails, do not read and
remove
                std::cout << "No emails to read.\n\n";
```

```
            } else {
                Email top = emailHeap.getMax(); //get the top email
                emailHeap.extractMax(); //take it out of the list using
extractmax function
            }

        } else if (command == "COUNT") { //tracks how many email is in the
list
            std::cout << "There are " << emailHeap.size() << " Emails to
read.\n\n";
        }

                    }

//main function
// main function takes in user inputted command list file
//checks if it exists,
// and runs through each command here.
//once done, close file and end

//no input, outputs 0 to end main or 1 if an error
int main() { //main function
    std::string filename; //user file
    std::cout << "Enter the command text file name: ";
    std::cin >> filename; //takes in user input for file

    std::ifstream file(filename);
    if (!file.is_open()) { //checks if file can be opened
        std::cerr << "Error: Could not open file " << filename <<
std::endl;
        return 1; //end early if cannot
    }

    EmailMaxHeap emailHeap; // creates email heap
    long long globalSequenceCounter = 0; //global counter
    std::string line; //for the lines of the user files

    while (std::getline(file, line)) { // while there are sill lines in
the file
        if (line.empty()) continue; // skip if nothing in that line
```

```
        std::stringstream ss(line); //get line as stream for individual
parts (ss)
        std::string command; // gets command from first part
        ss >> command; // get command

        //call command function with all variables needed to perform
commands
        processCommand(command, ss, emailHeap, globalSequenceCounter);

    } //end of while loop

    file.close(); //always close file when done
    return 0; //end main
} // end of main
```